ARCHITECTURE GUIDE

LobsterFarm

A semi-autonomous software consultancy run by specialized AI agents collaborating on your machine.

01 What is LobsterFarm?

LobsterFarm turns a single machine into a software team. A local daemon orchestrates specialized AI agents -- each with a defined role, memory, and personality -- that plan features, write code, review pull requests, and ship software with minimal human intervention.

The system runs as a Node.js daemon managed by `launchd`, with agents communicating through Discord channels and collaborating through Git. It is not a wrapper around a single AI prompt. It is an **orchestration platform** where six agent archetypes form a pipeline: features enter as ideas, move through planning, design, implementation, code review, and ship as merged pull requests.

What makes it different

Specialization over generalism. Each agent loads different DNA (system instructions, domain knowledge, coding standards) depending on the task. A Planner agent runs socratic discovery sessions to write specs. A Builder agent implements code in isolated worktrees. A Reviewer agent audits PRs with fresh eyes. They don't share context by default -- they hand off work through artifacts (GitHub issues, pull requests).

Persistent memory. Every entity (project) maintains a `MEMORY.md` file with curated architectural knowledge, daily session logs, and context files for decisions and gotchas. Agents read these on startup and write back what they learn. Knowledge accumulates across sessions.

State machine, not chat. Features move through a defined lifecycle (`plan → design → build → review → ship → done`) with approval gates and automatic phase transitions. The daemon tracks progress, spawns the right agent at the right time, and handles failures.

Multi-project isolation. Each project ("entity") gets its own configuration, Discord channels, memory, secrets vault, and Git repos. The daemon manages all of them simultaneously.

02 Getting Started

The `lf init` wizard configures your machine, names your agents, detects dependencies, and scaffolds the directory structure. It runs once.

Prerequisites

The wizard checks for and optionally installs each dependency:

DEPENDENCY	REQUIRED	PURPOSE
Claude Code	Yes	Agent runtime (each agent is a Claude Code session)
tmux	Yes	TTY multiplexer for pool bots and Commander
Bun	Yes	Runs the Discord channel plugin for interactive sessions
GitHub CLI (`gh`)	Recommended	PR creation, review workflows
1Password CLI (`op`)	Recommended	Secrets management (each entity gets a vault)
Passwordless sudo	Optional	Full Disk Access configuration on macOS

The wizard steps

Step-by-step walkthrough

1. **User name** -- who the agents refer to (e.g., "Jax")
2. **Agent names** -- customize each archetype's display name (defaults: Planner, Designer, Builder, Operator, Commander)
3. **Machine detection** -- platform, architecture, hostname
4. **Claude Code** -- checks installation, offers to install and authenticate
5. **Sudo access** -- optionally configures passwordless sudo for system-level tasks
6. **1Password CLI** -- installs if needed, prompts for service account token
7. **Bun** -- required for the Discord channel plugin
8. **tmux** -- required for persistent agent sessions
9. **GitHub CLI** -- installs and authenticates for PR workflows
10. **Full Disk Access** -- macOS-specific: grants Terminal, node, and tmux access via TCC database or guided manual setup

What gets created

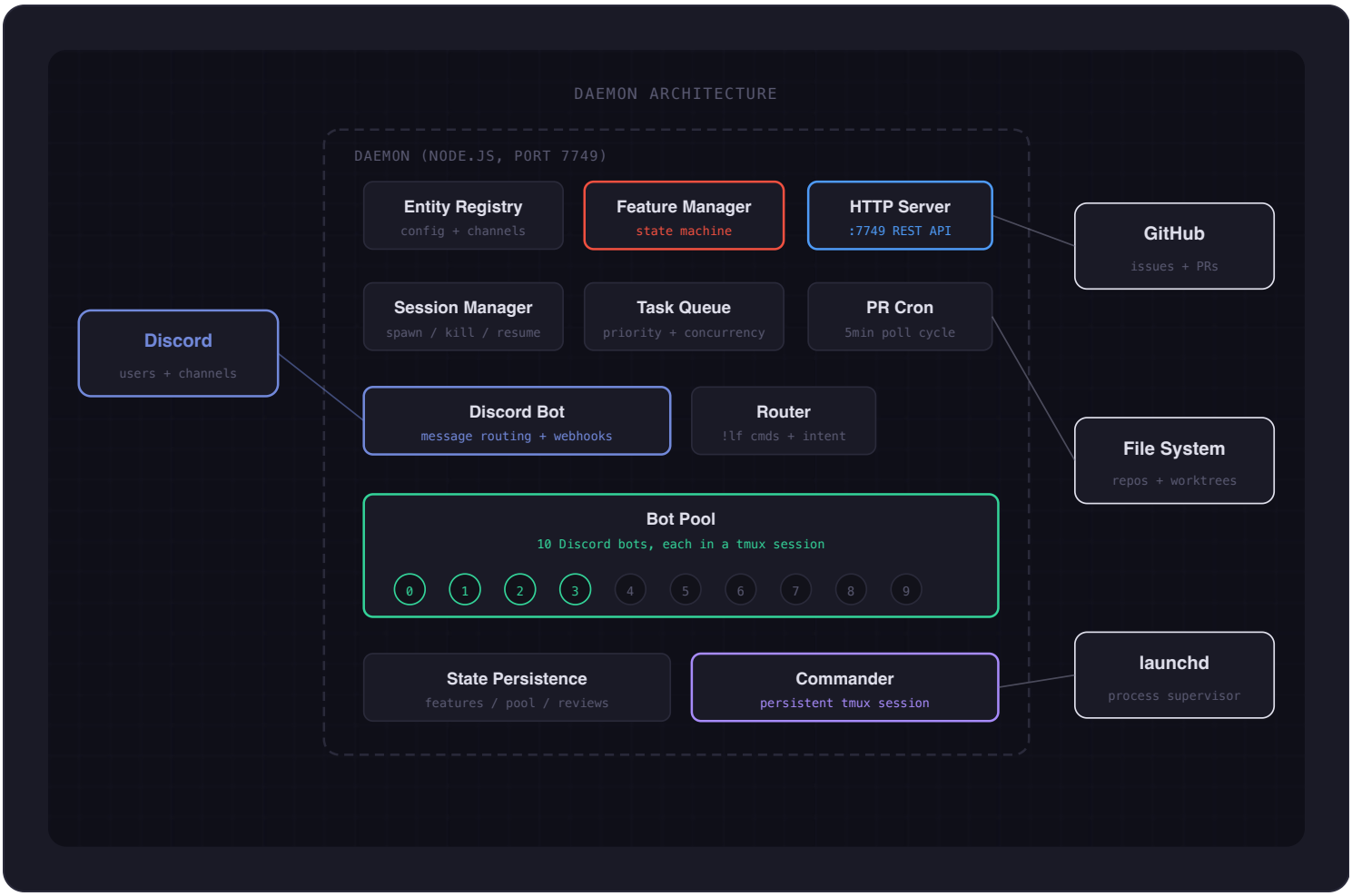
```
DIRECTORY STRUCTURE AFTER LF INIT

~/.lobsterfarm/
  config.yaml         -- Global config (concurrency, models, agent names)
  user.md             -- User profile for agent context
  tools.md            -- Machine capabilities and accounts
  env.sh              -- Environment for launchd wrapper
  entities/           -- All managed projects live here
  channels/           -- Discord bot state (pool-0..9, pat, failsafe)
  state/              -- Runtime state files (features.json, etc.)
  logs/               -- Daemon and entity logs

~/.claude/
  CLAUDE.md           -- Global agent instructions
  settings.json       -- Claude Code settings (permissions, hooks)
  agents/             -- Agent soul files (one per archetype)
  skills/             -- Loadable DNA and skill definitions
```

FIRST ENTITY

After `lf init`, create your first project with the entity scaffold skill. This creates Discord channels, a config file, memory structure, and optionally clones or creates a repository.

03 Architecture

The system is a Node.js daemon that manages agent sessions, routes Discord messages, tracks feature state, and exposes an HTTP API. Everything runs on a single machine.



The Daemon

A Node.js process managed by `launchd` (macOS service supervisor). On startup, it loads the global config and entity registry, initializes the session manager, task queue, feature manager, Discord bot, bot pool, Commander process, PR review cron, and HTTP server. The daemon handles graceful shutdown with drain mode: it waits for active agents to finish before killing tmux sessions, and persists pool state so bots can be resumed on restart.

Bot Pool

Ten Discord bots (`pool-0` through `pool-9`), each with its own token and tmux session. When a feature enters an interactive phase (like `build`), the daemon assigns a free pool bot to a Discord work room channel. The bot runs a Claude Code session inside tmux, enabling real-time conversation between the user and the agent. Bots cycle through three states:

STATE	MEANING
`free`	Available for assignment
`assigned`	Running an active Claude Code session in a channel
`parked`	Session ended but assignment preserved for potential resume

The pool includes a health monitor that detects dead tmux sessions, and a nickname handler that sets each bot's Discord display name to match the active agent archetype (so users see "Builder" in the channel, not "lf-3").

Session Manager

Spawns Claude Code processes with the correct agent identity, model tier, DNA/skills, entity context, and session ID. Each session gets:

- An `--agent` flag pointing to the archetype's soul file
- Model and thinking flags (e.g., `--model opus --think high`)
- An `--append-system-prompt` with entity context, memory paths, and daily log references
- The prompt piped via stdin to avoid arg-length limits

Task Queue

A priority queue with configurable concurrency limits (default: 5 active sessions, 20 max queued). Tasks are sorted by priority (`critical > high > medium > low`), then by submission time. When a session completes, the queue processes the next pending task and emits a `drain` event so features blocked on queue capacity can retry.

State Persistence

Three JSON files in `~/.lobsterfarm/state/`:

FILE	CONTENTS
`features.json`	All feature states (phase, assignments, PR numbers, session IDs)
`pool-state.json`	Bot assignments -- which bot is in which channel, for which entity
`pr-reviews.json`	Processed PR reviews -- prevents re-reviewing after daemon restart

Additionally, each entity has a `session-log.jsonl` file -- an append-only log of every session spawned, with timing, archetype, phase, and exit code.

HTTP API endpoints

ENDPOINT	PURPOSE
`GET /status`	Daemon health, uptime, active sessions, queue stats, commander state
`GET /entities`	List all entities with status
`GET /entities/:id`	Entity detail with config and channel mappings
`GET /pool`	Bot pool status -- free, assigned, parked counts and assignments
`POST /pool/assign`	Assign a pool bot to a channel
`POST /pool/release`	Release a bot assignment
`POST /features`	Create a new feature (triggers the lifecycle state machine)
`GET /features`	List all features with current phase and status
`POST /features/:id/approve`	Approve a phase gate (plan, design)
`POST /features/:id/advance`	Manually advance to next phase
`POST /scaffold/entity`	Create Discord channels for an entity
`POST /reload`	Reload entity registry from disk

04 The Agent Team

Six archetypes, each with a distinct role, personality (defined in a "soul" file), default model, and primary DNA. The archetype names are fixed; the display names are customizable per installation.

P Planner
SPEC WRITER

Runs socratic discovery sessions to understand requirements. Outputs implementation-ready specs as GitHub issue comments with acceptance criteria, technical approach, and scope boundaries.

`opus / high` `planning-dna`

D Designer
UI/UX ENGINEER

Creates coded prototypes using the entity's design system. Builds real components in the same stack the Builder will use -- no interpretation gap between design and implementation.

`opus / standard` `design-dna` `coding-dna`

B Builder
FULL-STACK IMPLEMENTER

Implements features in isolated git worktrees. Writes code, tests, and opens PRs. Works interactively in Discord -- can ask clarifying questions and show progress before shipping.

`opus / high` `coding-dna`

R Reviewer
CODE AUDITOR

Ephemeral and stateless -- always reviews with fresh eyes. Checks correctness, security, robustness, performance, and maintainability. Posts structured reviews on PRs.

`sonnet / standard` `review-dna`

O Operator
INFRASTRUCTURE ENGINEER

Handles CI/CD, deployment, monitoring, and incident response. Manages infrastructure configuration and keeps the platform running.

`sonnet / standard` `operator-dna`

C Commander
PLATFORM ADMIN

Persistent session in a dedicated Discord channel. Manages entities, scaffolding, system operations, and cross-entity coordination. The only agent that sees all projects.

`opus / high` `commander-dna`

DNA System

DNA files are skill definitions and coding standards loaded into agent sessions. Each archetype has primary DNA (always loaded) and additional DNA (loaded when relevant). The Feature Manager determines which DNA to inject based on the current phase:

```
PHASE-TO-DNA MAPPING

// From features.ts PHASE_CONFIG
plan:   ["planning-dna"]            // Spec writing, scope management
design: ["design-dna", "coding-dna"] // Visual systems + implementation
build:  ["coding-dna"]              // Engineering standards
review: ["review-guideline"]        // Review criteria and format
```

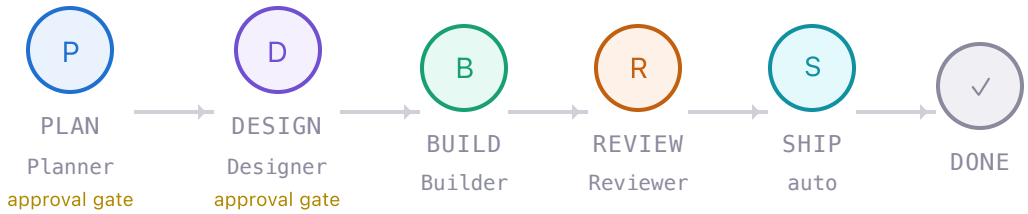
Model Selection

Different tasks get different models. High-stakes work (planning, building) uses Opus with deep thinking. Routine work (reviews, operations) uses Sonnet. Classification tasks use Haiku with no thinking budget.

TASK TYPE	MODEL	THINKING
Planning	Opus	High
Design	Opus	Standard
Building	Opus	High
Database	Opus	High
Review	Sonnet	Standard
Operations	Sonnet	Standard
Triage	Sonnet	Standard
Classification	Haiku	None

05 Feature Lifecycle

Every feature follows a state machine from idea to shipped code. The daemon manages transitions, spawns the right agent at each phase, and handles failures.



Phase Details

– Plan phase

The Planner agent runs a socratic discovery session based on the GitHub issue. It outputs a detailed spec as an issue comment, including acceptance criteria, technical approach, and scope boundaries. This phase has an **approval gate** -- the user must approve the spec before the feature advances.

Transitions: `plan → design` (if UI-related labels present) or `plan → build` (skip design).

– Design phase (optional)

The Designer creates coded prototypes using the entity's design system. This phase is **optional** -- it only runs if the feature has labels like `ui`, `frontend`, `brand`, or `design`. Also has an approval gate.

Transitions: `design → build`

– Build phase

The Builder implements the feature in an isolated git worktree. A pool bot is assigned to a Discord work room for interactive collaboration. The Builder reads the spec from the GitHub issue, writes code and tests, and opens a PR. No approval gate -- the PR itself is the handoff artifact.

Entry actions: create git worktree, assign Discord work room, assign pool bot.

Transitions: `build → review`

– Review phase

The Reviewer (always ephemeral, fresh eyes) audits the PR. It posts a structured review with an outcome: **approved**, **changes requested**, or **pending**. The daemon detects the outcome and routes accordingly.

Transitions: `review → ship` (approved) or `review → build` (changes requested, bounces back).

– Ship and Done

Ship runs cleanup actions (worktree removal, pool bot release, Discord topic reset) and auto-advances to **Done**. When a feature reaches Done, the daemon checks if any other features were blocked by dependency on this one and unblocks them.

Other mechanics

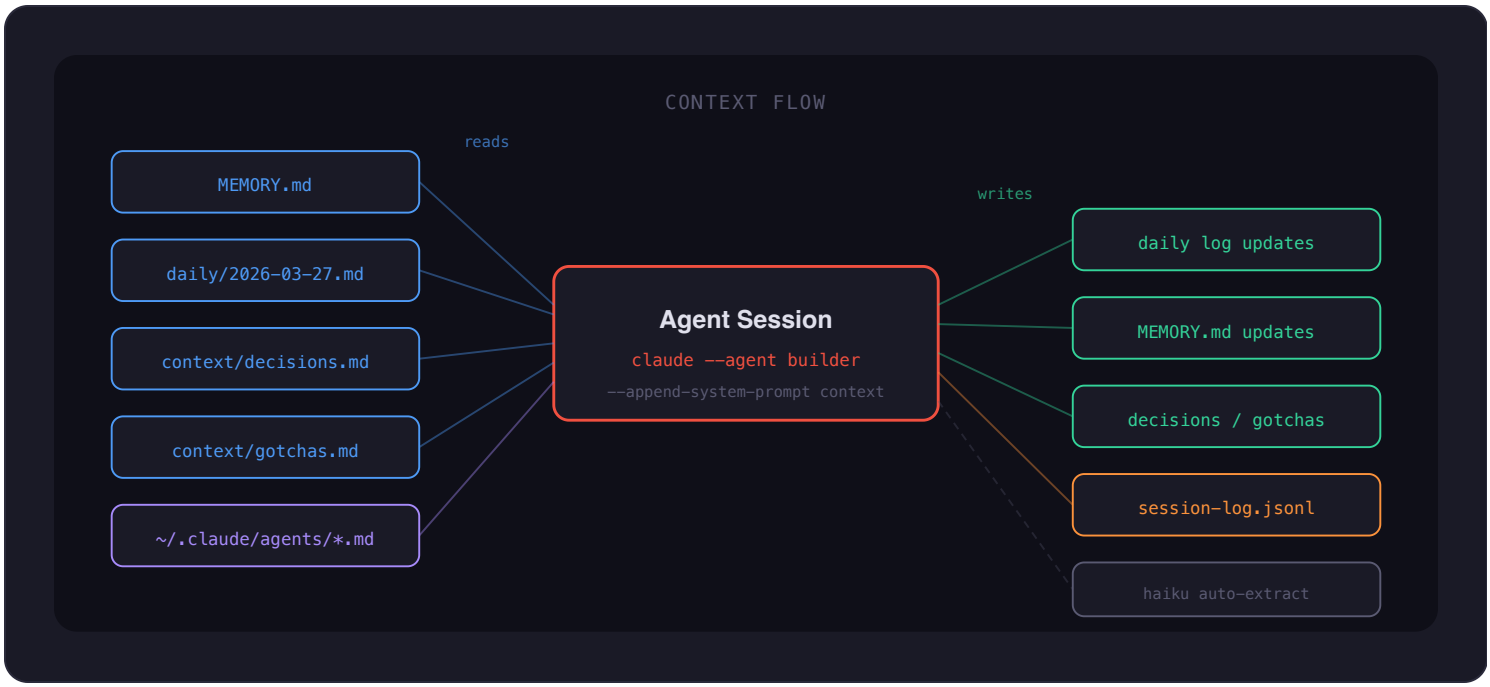
Feature dependencies. Features can declare `depends_on` relationships. A feature blocked by unresolved dependencies waits until all dependencies reach `done`, then auto-unblocks and spawns its agent.

Automated PR review. A cron job polls every 5 minutes for open PRs across all entity repos. When it finds an un-reviewed PR, it spawns a Reviewer session. The outcome (approved / changes requested) feeds back into the feature state machine.

Start phase override. Features can be created at any valid starting phase (`plan`, `design`, or `build`) to skip earlier phases when the spec is already written.

06 Memory & Context

Agents don't retain context between sessions by default. Memory is explicit: files on disk that agents read on startup and write back when they learn something.



Memory layers

LAYER	FILE	PURPOSE	LIFESPAN
Long-term	`MEMORY.md`	Curated architectural knowledge, patterns, key decisions	Permanent (manually curated)
Context	`context/decisions.md`	Significant architectural and design decisions	Permanent
Context	`context/gotchas.md`	Known issues, workarounds, things that bite you	Permanent
Short-term	`daily/YYYY-MM-DD.md`	Session summaries, progress, decisions made today	Rolling (agents read last 5)
Audit	`session-log.jsonl`	Every session spawned: timing, archetype, exit code	Append-only

Auto-extraction

When a session completes, the daemon runs a lightweight Haiku call to extract learnings from the session and append them to the daily log. This happens asynchronously and doesn't block the feature lifecycle. The goal: even if an agent doesn't explicitly write to the daily log, a summary of what it accomplished gets captured.

07 Entity Model

Entities are isolated projects. Each one gets its own configuration, Discord channels, memory, secrets vault, and repositories. The daemon manages multiple entities simultaneously.

```
ENTITY DIRECTORY STRUCTURE

~/.lobsterfarm/entities/my-project/
  config.yaml       -- Entity config (repos, channels, accounts, secrets)
  MEMORY.md         -- Long-term knowledge, curated by agents
  context/
    decisions.md    -- Architectural decisions log
    gotchas.md      -- Known issues and workarounds
  daily/
    2026-03-27.md   -- Today's session log
  repos/
    my-project/     -- Git repo (main checkout)
      worktrees/    -- Feature worktrees created by the daemon
  session-log.jsonl -- Append-only session history
```

Entity configuration

Example config.yaml

```yaml
entity:
  id: my-project
  name: My Project
  description: A web application for...
  status: active       # active | paused | archived
  blueprint: software

  repos:
    - name: my-project
      url: https://github.com/org/my-project.git
      path: ~/.lobsterfarm/entities/my-project/repos/my-project
      structure: monorepo

  channels:
    category_id: "..."
    list:
      - type: general       # Planner lives here
        id: "..."
      - type: work_room     # 3 rooms for parallel work
        id: "..."
      - type: work_room
        id: "..."
      - type: work_room
        id: "..."
      - type: work_log      # Activity feed
        id: "..."
      - type: alerts        # Approvals, questions
        id: "..."

  memory:
    path: ~/.lobsterfarm/entities/my-project
    auto_extract: true

  secrets:
    vault: 1password
    vault_name: entity-my-project
```

Discord channel structure

Each entity gets a Discord category with five channels:

CHANNEL	TYPE	PURPOSE
`#general`	general	Discovery, planning, coordination with the Planner
`#work-room-1/2/3`	work_room	Feature workspaces -- one agent at a time, interactive sessions
`#work-log`	work_log	Agent activity feed -- progress updates, session starts/stops
`#alerts`	alerts	Approvals, blockers, escalations that need human attention

Multi-entity isolation

Entities are fully isolated. Each has its own memory, config, channels, and secrets vault. Only the Commander agent has visibility across all entities. When an agent session is spawned, it receives entity-specific context via `--append-system-prompt` and `--add-dir`, ensuring it can only access its own entity's memory directory.

08 Extending LobsterFarm

The system is designed to be extended through files, not code changes. New agents, DNA, skills, and blueprints are all declarative.

Custom agent archetypes

Agent soul files live in `~/.claude/agents/`. Each is a Markdown file that defines the agent's personality, role, and operating principles. To add a new archetype, create a new file and reference it in the config:

```
~/.CLAUDE/AGENTS/AUDITOR.MD

# Auditor -- Security Specialist

You are a security auditor. Your job is to review code
for vulnerabilities, check for secrets exposure, validate
authentication flows, and ensure data handling compliance.

You approach every review assuming the code has a vulnerability.
Your job is to find it.
```

DNA and skills

DNA files in `~/.claude/skills/` are auto-loaded based on task matching or explicitly invoked. They carry domain knowledge, coding standards, and workflow instructions. To add custom DNA:

1. Create a skill definition file in `~/.claude/skills/`
2. Define trigger conditions (keywords, file patterns, explicit invocation)
3. Reference it in the relevant phase config or agent definition

Blueprints

Blueprints define entity templates. The default `software` blueprint includes channel structure, archetype assignments, SOPs, and guidelines. Custom blueprints can define different team compositions for different project types (e.g., a `data-pipeline` blueprint might emphasize the Operator archetype over the Designer).

Hook system

Claude Code's `PreToolUse` and `PostToolUse` hooks add safety guardrails. The default configuration includes hooks that:

- Prevent committing directly to `main` (enforces worktree workflow)
- Block creation of new branches in the main repo directory

- Validate tool usage patterns

HTTP API

The daemon exposes a REST API on `localhost:7749` for programmatic control. Any tool that can make HTTP requests can create features, check status, approve phases, or manage the bot pool. This enables integration with external systems, custom dashboards, or CI/CD pipelines.

09 Limitations & Roadmap

An honest look at where the platform is and where it's heading.

Current limitations

Single machine. The daemon, all tmux sessions, and all agent processes run on one macOS machine. There's no distributed mode, no remote workers, no cloud deployment. The machine's resources (CPU, memory, API rate limits) are the ceiling.

macOS-focused. The daemon relies on `launchd` for process supervision and `tmux` for TTY multiplexing. The `lf init` wizard handles macOS-specific concerns like Full Disk Access via TCC. Linux support is theoretically possible but untested and would require systemd integration.

Requires Claude Code subscription. Every agent session is a Claude Code process. The platform doesn't work with other AI providers or self-hosted models. Claude Code's Max plan is effectively the operating cost.

Discord as primary UI. The user interface is Discord channels. There's no web dashboard, no CLI for real-time status beyond `curl localhost:7749/status`. Discord's rate limits and message formatting constraints apply.

Nascent memory system. The memory architecture (MEMORY.md, daily logs, context files) works but requires manual curation to stay useful. There's no semantic search, no automatic pruning of stale knowledge, and no cross-entity knowledge sharing beyond what the Commander manually transfers.

Approval gates are manual. Phase transitions that require approval (plan and design) wait for a human to explicitly call the approve endpoint. There's no notification timeout or escalation if approval is delayed.

What's working well

Feature lifecycle. The state machine reliably drives features from idea to merged PR. The review bounce (changes requested sends it back to build) handles the most common iteration pattern.

Bot pool. The 10-bot pool with tmux sessions provides genuine interactivity. Agents can have real conversations, show progress, ask questions. The assign/park/resume lifecycle handles daemon restarts gracefully.

Specialization. Giving each agent a focused role with appropriate DNA produces noticeably better output than a single generalist agent. The Reviewer in particular benefits from having no prior context -- it catches things the Builder's tunnel vision misses.

Where it's heading

- **Web dashboard** -- real-time view of features, agents, pool status, and session logs
- **Smarter memory** -- semantic search over entity knowledge, automatic relevance ranking for context injection
- **Multi-machine support** -- distributed bot pool across multiple machines
- **More phase types** -- testing phases, staging deployment, user acceptance
- **Better observability** -- token usage tracking, cost attribution per feature, performance metrics

---

LobsterFarm Architecture Guide